Tools, models & methods

Bangladesh essential-commodity prices: data pipeline, analysis, forecasting and a daily-updated bilingual website

Live: pricepulsebd.vercel.app · Atiqul Islam Chowdhury · November 2026

What this is

PricePulse BD collects the Trading Corporation of Bangladesh (TCB) daily retail price sheets for Dhaka, cleans 1,395 reports into a daily panel of 61 essential commodities (May 2020 – today), analyses what moved and why, tests whether prices can be forecast, and publishes everything as a bilingual (English/Bangla) website that rebuilds itself every day. Everything runs on free tools: no paid APIs, no servers.

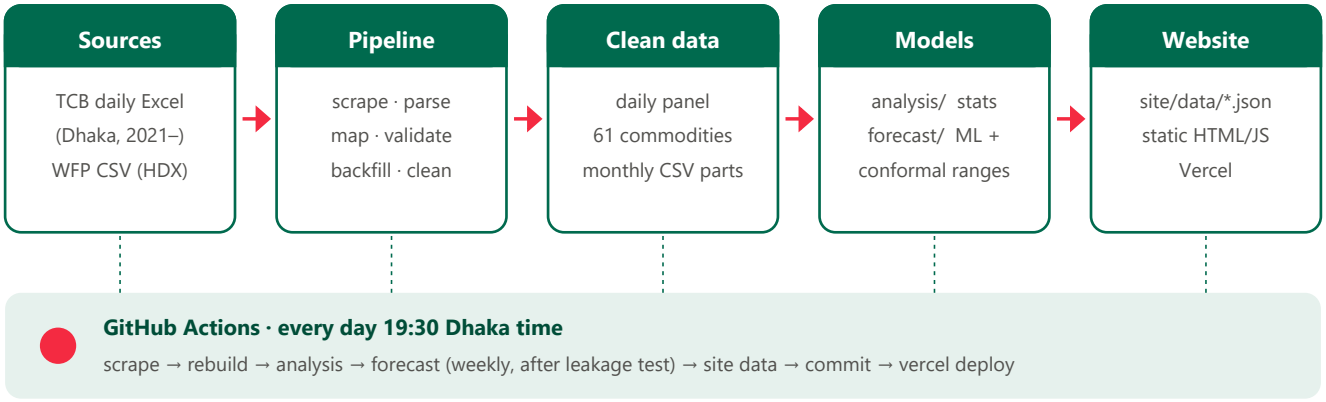


Figure 1. End-to-end architecture. Each box is a folder in the repository.

1. Tools

Layer	Tool	Used for
Data & ingestion	Python 3.11, pandas, NumPy, PyArrow, requests, openpyxl, certifi	Incremental crawl of the TCB archive, Excel parsing, tidy panels, partitioned storage; TLS with a bundled Sectigo intermediate certificate
Statistics & ML	SciPy, statsmodels, scikit-learn	Permutation, Mann-Whitney and Granger tests; Diebold-Mariano (own code, Newey-West); HistGradientBoostingRegressor (mean, median, 10% / 90% quantile)
Report & notebooks	Matplotlib, nbformat, nbclient	16 figures in a validated colour-blind-safe palette; notebooks generated from code and executed
Website	HTML, CSS, vanilla JavaScript, Noto Sans Bengali	No framework or build step; hand-written SVG charts; Intl formatting for Bangla digits and dates
Testing	Chrome headless, Node.js	Screenshots (desktop, 390 px, dark mode); DevTools-protocol script clicking every control (18 checks)
Automation & hosting	GitHub Actions, Vercel CLI 62	Daily scrape → analysis → forecast → commit → deploy to pricepulsebd.vercel.app
Dev tools	git, GitHub CLI, Claude Code	Version control; repo, secrets and workflow runs; AI pair-programming assistant under the author's direction

2. Data pipeline

`pipeline/`: scrape_tcb → parse_tcb → build_dataset (+ fetch_wfp). Incremental: a day with nothing new costs one listing request.

Problem found in the source	Engineering response
tcb.gov.bd serves an incomplete TLS certificate chain	Ship the public Sectigo DV R36 intermediate and verify against it; verification never disabled
36 archive entries (29 Jan – 8 Mar 2025) link to the wrong file; one file is zero-filled	Detected and logged as source problems (`tcb_parse_failures.csv`)
Header row moves and repeats mid-sheet before sugar, eggs, dates, salt	Header located by content; repeated headers skipped, not treated as end of table
Same product under many labels (local garlic: 8 spellings)	81 raw labels mapped to 61 commodities; variants verified never to co-occur on one report
Two Bangla letters (U+09DF, U+09DC) stored either precomposed or as base letter + nukta	Unicode NFC normalisation before any matching (also in the site's search box)
Missing days (2022 has only 64 reports)	Back-filled from each sheet's 1-week / 1-month / 1-year columns, which match the original reports 99.6–99.8% of the time; every value keeps its `source`
Data-entry typos (cumin Tk 4,000 for Tk 400; swapped min/max)	Explicit rules (×10 digit fix, swap, isolated spike-and-revert), each change logged

Result: **105,094** daily rows, 61 commodities. External check: monthly TCB prices track the independent WFP survey (level correlation 0.85–0.97).

3. Analysis methods

Question	Method	Result
What got expensive?	Jevons index (geometric mean of price relatives), FY2020-21 = 100; one-sided permutation test (20,000 shuffles) and Mann-Whitney	Import-dependent items +71% vs domestic +39% (median); p = 0.002
When do prices spike?	14-day log change scored by a robust z (median / MAD of the item's own trailing year); z ≥ 3.5 and rise ≥ 10%; flagged days merged into episodes	6/6 evaluable documented supply shocks detected within 0–5 days
Do festivals move prices?	Event study: run-up (days −14…−1 vs −60…−31) compared with placebo dates; permutation test; Benjamini-Hochberg FDR across items	q < 0.10 for 10 items (Ramadan), 9 (Eid-ul-Fitr), 7 (Eid-ul-Adha); meat rises, iftar staples do not
Which prices move together?	Correlation of weekly log changes, average-linkage clustering on 1 − r; Granger causality (2 lags)	Families: rice grades, edible oils, imported staples; imported onion does not lead local
When is an item cheap?	Seasonal profile: ratio to a centred 13-month moving average, averaged by calendar month (≥ 3 years per month)	e.g. onion cheapest in April, dearest in November

4. Forecasting

Target: the Thursday price 1–4 weeks ahead, as a log change. Point-in-time Thursday prices are used instead of weekly averages, because averaging a random walk creates spurious predictability.

Models and accuracy

Mean absolute error relative to naive (1.00 = same; above 1 = worse). 39,652 out-of-sample forecasts, 57 commodities, 2023-02-02 to 2026-11-05.

Model	Idea	1 wk	2 wk	3 wk	4 wk
Naive (shipped)	Next price = today's price	1.00	1.00	1.00	1.00
Exponential smoothing	ETS(A,N,N); alpha re-chosen at every origin	1.04	1.02	1.01	1.01
Gradient boosting, median	Pooled across items; quantile loss 0.5	1.01	1.01	1.02	1.04
Naive + drift	Add the item's mean weekly change, last year	1.12	1.12	1.12	1.13

Model	Idea	1 wk	2 wk	3 wk	4 wk
Gradient boosting, mean	Pooled; squared-error loss; 25 features	1.29	1.27	1.22	1.24
Seasonal naive	Repeat last year's change for the same weeks	1.83	1.73	1.67	1.63

Every alternative is significantly worse than naive (Diebold-Mariano, $p < 0.001$). Prices do not move in about 63% of weeks, and moves are nearly uncorrelated with the previous move, so the honest central forecast is today's price.

Prediction intervals

Interval method (nominal 80%)	1 wk	2 wk	3 wk	4 wk
Item's own past changes (10th–90th pct)	85.5%	83.4%	82.0%	81.7%
Quantile GBM, raw	80.1%	79.0%	76.3%	74.2%
Quantile GBM + conformal (shipped)	78.9%	80.6%	80.4%	80.8%
Quantile GBM + per-item conformal	78.9%	79.5%	79.1%	79.3%

Conformalized quantile regression (Romano et al., 2019): the raw quantile range is widened or narrowed by the 80th percentile of the previous 52 weeks' out-of-sample misses, measured in each item's own volatility units. It beats the historical-quantile baseline on interval score at every horizon ($p \le 0.032$).

Evaluation protocol

- **Rolling origin:** a forecast from every Thursday as if live; expanding training window; boosted models refit every 13 weeks on rows whose outcome was already known.
- **Pre-registered selection:** rules written in code before results were read — ship a model only if it beats naive with DM $p < 0.05$; ship the interval method with the best interval score among those within ±3 points of 80% coverage at every horizon.
- **Leakage test:** `tests/test_no_leakage.py` rebuilds every feature, baseline and interval with the future deleted and requires identical values.
- **Reproducibility:** multi-threaded boosting is not bit-identical across machines; a Linux re-run changed no conclusion but moved some single-item ranges by a few points.
- **Features (25):** past changes (1–52 weeks), distance from 8/26/52-week averages, 13/52-week volatility, weeks since last move, TCB min–max spread, category and basket trend, week of year, days to Ramadan / Eid.

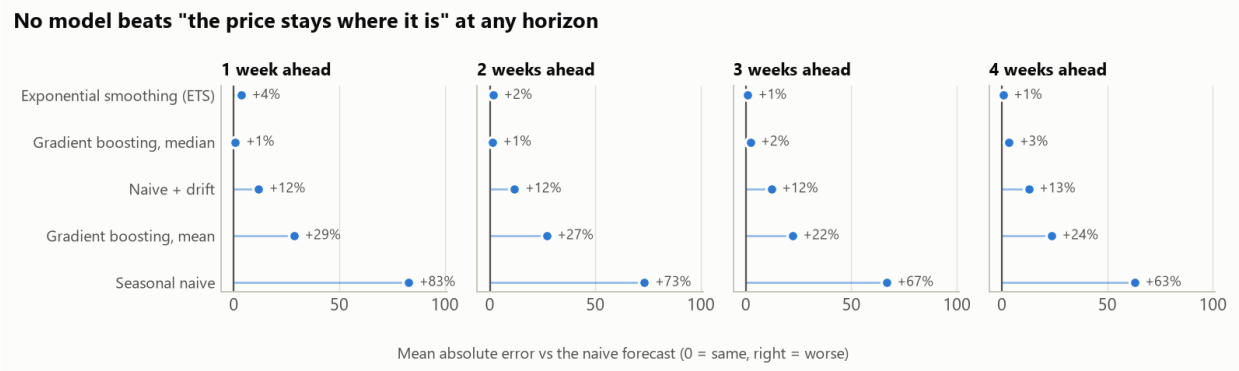


Figure 2. Forecast error relative to the naive forecast, by horizon (lower is better; 0 = naive).

5. Website

A static page reading two JSON payloads written by `python -m analysis.site`: `app.json` (~106 KB, ~30 KB compressed) and one history file per item, fetched on demand.

Feature	Implementation notes
Price table	Search (NFC-normalised), category filter, sort, 6-month SVG sparklines
My bazaar	Editable monthly basket saved in localStorage; bill vs 1 month / 1 year; segmented spending bar; per-item drivers
Compare items	Up to 4 items indexed to 100; colours tied to the item, not its position
When prices move	Month-on-month heatmap, diverging blue–grey–red scale; 12 months on phones

Feature	Implementation notes
Item view	Min–max band, 4-week 80% range and its backtest coverage, typical-year bars, share link
Festivals	Effects with significance (filled = q < 0.10) and the average price path around Ramadan / Eid with the range across years
Quality	English/Bangla, dark mode, keyboard-operable charts with table views, validated colour-blind-safe palette (flag green/red kept to page chrome)

6. Automation and hosting

- Workflow `.github/workflows/daily.yml` runs at 19:30 Dhaka time; a day without a new TCB sheet commits and deploys nothing.
- New forecasts only when a newer Thursday arrives (`forecast/due.py`), after the leakage test passes. A full backtest takes about 3 minutes on GitHub's runners.
- Deployment: `vercel deploy --prod` from the workflow using the `VERCEL_TOKEN` repository secret; only `site/` is uploaded.
- Cost: about 5 minutes of Actions time per day, inside the free allowance; Vercel Hobby plan.

7. Reproduce

```
pip install -r requirements.txt
python pipeline/run_all.py            # data
python -m analysis.run                # analysis, figures (~2 min)
python -m forecast.run                # backtest + forecast (~3–8 min)
python tests/test_no_leakage.py
python -m analysis.site               # website data
python notebooks/build_03_forecasting.py # executed notebook
```

8. Limitations

- Dhaka only, about six years: seasonal patterns are seen five or six times, each in a different shock year.
- TCB quotes a min–max range, not transaction prices; midpoints move in steps.
- No exchange-rate, world-price, duty or weather inputs; policy shocks cannot be anticipated.
- Festival effects rest on 5–7 occurrences, and recent Ramadans coincided with harvest season.
- Interval coverage is about 80% on average; individual items range from roughly 65–70% to 98%.